

▼ Pertemuan 3 — Data Preparation & KNN Imputation

Notebook ini berisi:

1. **Upload & tampilkan gambar** dari Orange/PostgreSQL
2. **Load dataset** Iris dan Bank
3. **Identifikasi missing value** dan statistik deskriptif
4. **KNN Imputation** pada data numerik (Iris) — perhitungan manual + kode
5. **KNN Imputation** pada data campuran (Bank) — perhitungan manual + kode
6. **Visualisasi** tabel jarak dan hasil imputasi

Petunjuk Penggunaan di Google Colab

Sebelum menjalankan notebook ini, upload file-file berikut:

Dataset:

- `IRIS.csv` — dataset Iris (150 baris)
- `bank.csv` — dataset Bank Marketing (239 baris)

Gambar dari folder `Assets/Pertemuan3/`:

- `DataIrisOrangePengukuranJarak.png`
- `Gambar Csv ke PostgreSQL.png`
- `SebelumScalling.png`
- `SesudahScalling.png`

Jalankan cell di bawah untuk mengupload semua file sekaligus.

▼ 1. Upload File yang Dibutuhkan

```
1 from google.colab import files
2 import os
3
4 print("=" * 60)
5 print("UPLOAD FILE")
6 print("=" * 60)
7 print("Silakan upload file berikut (bisa pilih sekaligus):")
8 print(" 1. IRIS.csv")
9 print(" 2. bank.csv")
10 print(" 3. DataIrisOrangePengukuranJarak.png")
11 print(" 4. Gambar Csv ke PostgreSQL.png")
12 print(" 5. SebelumScalling.png")
13 print(" 6. SesudahScalling.png")
14 print()
15
16 uploaded = files.upload()
17
18 print("\nFile yang berhasil diupload:")
19 for fname in uploaded.keys():
20     print(f"  ✓ {fname} ({len(uploaded[fname])} bytes)")
```

```
=====
UPLOAD FILE
=====
Silakan upload file berikut (bisa pilih sekaligus):
1. IRIS.csv
2. bank.csv
3. DataIrisOrangePengukuranJarak.png
4. Gambar Csv ke PostgreSQL.png
5. SebelumScalling.png
6. SesudahScalling.png
```

IRIS.csv

IRIS.csv(text/csv) - 4617 bytes, last modified: 3/9/2026 - 100% done
Saving IRIS.csv to IRIS (1).csv

File yang berhasil diupload:
✓ IRIS (1).csv (4617 bytes)

```

1 from google.colab import files
2 import os
3
4 print("=" * 60)
5 print("UPLOAD FILE")
6 print("=" * 60)
7 print("Silakan upload file berikut (bisa pilih sekaligus):")
8 print("  1. IRIS.csv")
9 print("  2. bank.csv")
10 print("  3. DataIrisOrangePengukuranJarak.png")
11 print("  4. Gambar Csv ke PostgreSQL.png")
12 print("  5. SebelumScalling.png")
13 print("  6. SesudahScalling.png")
14 print()
15
16 uploaded = files.upload()
17
18 print("\nFile yang berhasil diupload:")
19 for fname in uploaded.keys():
20     print(f"  ✓ {fname} ({len(uploaded[fname])} bytes)")

```

```

=====
UPLOAD FILE
=====
Silakan upload file berikut (bisa pilih sekaligus):
  1. IRIS.csv
  2. bank.csv
  3. DataIrisOrangePengukuranJarak.png
  4. Gambar Csv ke PostgreSQL.png
  5. SebelumScalling.png
  6. SesudahScalling.png

```

Choose Files 4 files

bank.csv(text/csv) - 20240 bytes, last modified: 3/9/2026 - 100% done
BankMarketing.sql(n/a) - 6975 bytes, last modified: 3/9/2026 - 100% done
MelakukanDistancediOrange.png(image/png) - 195762 bytes, last modified: 3/9/2026 - 100% done
Sudah Menghubungkan ke Postgree.png(image/png) - 277357 bytes, last modified: 3/9/2026 - 100% done
 Saving bank.csv to bank.csv
 Saving BankMarketing.sql to BankMarketing.sql
 Saving MelakukanDistancediOrange.png to MelakukanDistancediOrange.png
 Saving Sudah Menghubungkan ke Postgree.png to Sudah Menghubungkan ke Postgree.png

File yang berhasil diupload:

- ✓ bank.csv (20240 bytes)
- ✓ BankMarketing.sql (6975 bytes)
- ✓ MelakukanDistancediOrange.png (195762 bytes)
- ✓ Sudah Menghubungkan ke Postgree.png (277357 bytes)

```

1  # Cek semua file yang dibutuhkan sudah ada
2  required_files = {
3      'dataset': ['IRIS.csv', 'bank.csv'],
4      'gambar': [
5          'DataIrisOrangePengukuranJarak.png',
6          'Gambar Csv ke PostgreSQL.png',
7          'SebelumScalling.png',
8          'SesudahScalling.png'
9      ]
10 }
11
12 print("Cek file yang tersedia:")
13 all_ok = True
14 for kategori, flist in required_files.items():
15     print(f"\n{kategori.upper()}:")
16     for f in flist:
17         exists = os.path.exists(f)
18         status = "✓" if exists else "X BELUM ADA"
19         print(f"  {status} {f}")
20         if not exists:
21             all_ok = False
22
23 if all_ok:
24     print("\n👍 Semua file sudah lengkap! Lanjutkan ke cell berikutnya.")
25 else:
26     print("\n⚠️ Ada file yang belum diupload. Jalankan ulang cell upload di atas.")

```

Cek file yang tersedia:

DATASET:
 ✓ IRIS.csv
 ✓ bank.csv

GAMBAR:
 X BELUM ADA DataIrisOrangePengukuranJarak.png
 X BELUM ADA Gambar Csv ke PostgreSQL.png

X BELUM ADA SebelumScaling.png
X BELUM ADA SesudahScaling.png

⚠ Ada file yang belum diupload. Jalankan ulang cell upload di atas.

2. Import Library

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from IPython.display import display, Image, Markdown
5
6 print("Library berhasil di-import.")
```

Library berhasil di-import.

3. Menampilkan Gambar dari Orange & PostgreSQL

Gambar-gambar berikut adalah bukti dari workflow Orange Data Mining dan hasil import data ke PostgreSQL.

3.1 Workflow Pengukuran Jarak Dataset Iris di Orange

Workflow ini menunjukkan alur pengukuran jarak di Orange:

- **CSV File Import** → memuat data Iris
- **Data Table** → menampilkan isi data
- Widget jarak: **Euclidean, Manhattan, Spearman, Hamming**
- **Distance Matrix** → menampilkan matriks jarak
- **Save Distance Matrix** → menyimpan hasil

```
1 # Tampilkan gambar workflow Orange
2 if os.path.exists('DataIrisOrangePengukuranJarak.png'):
3     img = plt.imread('DataIrisOrangePengukuranJarak.png')
4     fig, ax = plt.subplots(figsize=(14, 8))
5     ax.imshow(img)
6     ax.axis('off')
7     ax.set_title('Workflow Pengukuran Jarak Dataset Iris di Orange',
8                 fontsize=14, fontweight='bold')
9     plt.tight_layout()
10    plt.show()
11 else:
12    print("⚠ File DataIrisOrangePengukuranJarak.png belum diupload.")
```

⚠ File DataIrisOrangePengukuranJarak.png belum diupload.

3.2 Bukti Data CSV ke PostgreSQL

Dataset Iris telah berhasil dimasukkan ke PostgreSQL menggunakan query:

```
SELECT * FROM public.iris
```

Total data: **150 baris**.

```
1 # Tampilkan bukti data di PostgreSQL
2 if os.path.exists('Gambar Csv ke PostgreeSQL.png'):
3     img = plt.imread('Gambar Csv ke PostgreeSQL.png')
4     fig, ax = plt.subplots(figsize=(14, 8))
5     ax.imshow(img)
6     ax.axis('off')
7     ax.set_title('Bukti Data CSV Telah Dimasukkan ke PostgreSQL', fontsize=14, fontweight='bold')
8     plt.tight_layout()
9     plt.show()
10 else:
11    print("⚠ File 'Gambar Csv ke PostgreeSQL.png' belum diupload.")
```

⚠ File 'Gambar Csv ke PostgreeSQL.png' belum diupload.

3.3 Histogram Fitur Iris — Sebelum dan Sesudah Scaling

Sebelum scaling: skala antar fitur berbeda-beda (misal petal_length 1-7, sepal_width 2-4).

Sesudah scaling (StandardScaler): semua fitur dinormalisasi dengan mean ≈ 0 dan std ≈ 1 , sehingga skala setara untuk perhitungan jarak.

```
1 # Tampilkan histogram sebelum dan sesudah scaling
2 fig, axes = plt.subplots(1, 2, figsize=(20, 8))
3
4 # Sebelum Scaling
5 if os.path.exists('SebelumScaling.png'):
6     img1 = plt.imread('SebelumScaling.png')
7     axes[0].imshow(img1)
8     axes[0].set_title('Sebelum Scaling', fontsize=13, fontweight='bold')
9 else:
10     axes[0].text(0.5, 0.5, 'File belum diupload', ha='center', va='center', fontsize=14)
11     axes[0].set_title('Sebelum Scaling (belum tersedia)', fontsize=13)
12 axes[0].axis('off')
13
14 # Sesudah Scaling
15 if os.path.exists('SesudahScaling.png'):
16     img2 = plt.imread('SesudahScaling.png')
17     axes[1].imshow(img2)
18     axes[1].set_title('Sesudah Scaling (StandardScaler)', fontsize=13, fontweight='bold')
19 else:
20     axes[1].text(0.5, 0.5, 'File belum diupload', ha='center', va='center', fontsize=14)
21     axes[1].set_title('Sesudah Scaling (belum tersedia)', fontsize=13)
22 axes[1].axis('off')
23
24 plt.suptitle('Perbandingan Histogram Fitur Iris: Sebelum vs Sesudah Scaling', fontsize=15, fontweight='bold', y=1.02)
25 plt.tight_layout()
26 plt.show()
```

Perbandingan Histogram Fitur Iris: Sebelum vs Sesudah Scaling

Sebelum Scaling (belum tersedia)

Sesudah Scaling (belum tersedia)

File belum diupload

File belum diupload

4. Load Dataset Iris

```
1 iris = pd.read_csv('IRIS.csv')
2 print(f"Dataset Iris: {iris.shape[0]} baris, {iris.shape[1]} kolom")
3 print(f"Kolom: {list(iris.columns)}")
4 iris.head(10)
```

Dataset Iris: 150 baris, 5 kolom
 Kolom: ['sepal_length', 'sepal_width', 'petal_length', 'petal_width', 'species']

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa
5	5.4	3.9	1.7	0.4	Iris-setosa
6	4.6	3.4	1.4	0.3	Iris-setosa
7	5.0	3.4	1.5	0.2	Iris-setosa
8	4.4	2.9	1.4	0.2	Iris-setosa
9	4.9	3.1	1.5	0.1	Iris-setosa

Langkah berikutnya: [New interactive sheet](#)

5. Identifikasi Missing Value & Statistik Deskriptif — Iris

```
1 # Cek missing value
2 print("Missing value per kolom:")
3 print(iris.isnull().sum())
4 print(f"\nTotal missing value: {iris.isnull().sum().sum()}")
5 print("\n→ Dataset Iris TIDAK memiliki missing value.")
6 print(" Oleh karena itu, akan dibuat missing value BUATAN untuk simulasi KNN.")
```

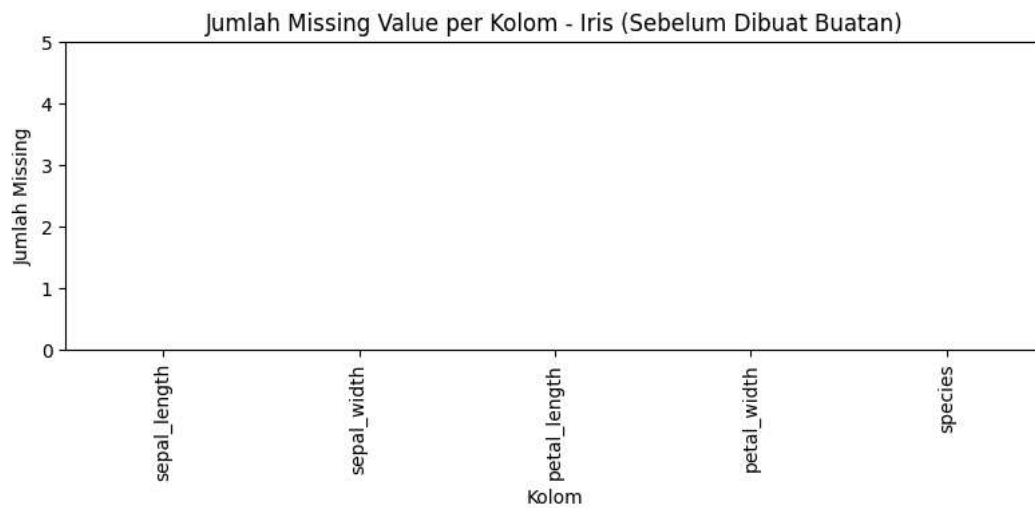
Missing value per kolom:

```
sepal_length    0
sepal_width     0
petal_length    0
petal_width     0
species         0
dtype: int64
```

Total missing value: 0

→ Dataset Iris TIDAK memiliki missing value.
 Oleh karena itu, akan dibuat missing value BUATAN untuk simulasi KNN.

```
1 # Grafik cek missing value Iris
2 missing_count = iris.isnull().sum()
3
4 plt.figure(figsize=(8, 4))
5 missing_count.plot(kind='bar', color='steelblue')
6 plt.title('Jumlah Missing Value per Kolom - Iris (Sebelum Dibuat Buatan)', fontsize=12)
7 plt.xlabel('Kolom')
8 plt.ylabel('Jumlah Missing')
9 plt.ylim(0, 5)
10 plt.tight_layout()
11 plt.show()
```



```

1 # Statistik deskriptif Iris
2 numeric_cols = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width']
3 print("Statistik Deskriptif - Dataset Iris:")
4 iris[numeric_cols].describe().T

```

Statistik Deskriptif - Dataset Iris:

	count	mean	std	min	25%	50%	75%	max
sepal_length	150.0	5.843333	0.828066	4.3	5.1	5.80	6.4	7.9
sepal_width	150.0	3.054000	0.433594	2.0	2.8	3.00	3.3	4.4
petal_length	150.0	3.758667	1.764420	1.0	1.6	4.35	5.1	6.9
petal_width	150.0	1.198667	0.763161	0.1	0.3	1.30	1.8	2.5

```

1 # Frekuensi tiap kelas
2 print("Frekuensi tiap species:")
3 print(iris['species'].value_counts())
4 print(f"\nTotal: {len(iris)} baris")

```

Frekuensi tiap species:

```

species
Iris-setosa      50
Iris-versicolor  50
Iris-virginica   50
Name: count, dtype: int64

```

Total: 150 baris

6. KNN Imputation — Dataset Iris (Data Numerik)

Konsep

Dataset Iris seluruhnya numerik (kecuali `species`), sehingga jarak dihitung dengan **Euclidean Distance**.

Langkah:

1. Buat missing value buatan pada **baris 5, kolom `petal_width`**
2. Hitung jarak Euclidean ke semua baris lain (**tanpa** kolom `petal_width`)
3. Ambil **k=3 tetangga terdekat**
4. Isi missing value = **rata-rata `petal_width`** dari 3 tetangga (KNN Regression)

Rumus Euclidean Distance:

$$d(i, j) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + \dots + (x_p - y_p)^2}$$

Aturan: Kolom yang memiliki missing value **tidak diikutkan** dalam perhitungan jarak.

```

1 # ===== MEMBUAT MISSING VALUE BUATAN =====
2 target_idx = 5
3 col_missing = 'petal_width'
4 nilai_asli_iris = iris.loc[target_idx, col_missing]
5
6 print(f"Data asli baris {target_idx}:")

```

```

7 print(iris.loc[[target_idx]])
8 print(f"\nNilai asli {col_missing}: {nilai_asli_iris}")
9
10 # Buat copy dan hilangkan nilai
11 iris_knn = iris.copy()
12 iris_knn.loc[target_idx, col_missing] = np.nan
13
14 print(f"\nData baris {target_idx} setelah dibuat missing:")
15 print(iris_knn.loc[[target_idx]])

```

Data asli baris 5:

	sepal_length	sepal_width	petal_length	petal_width	species
5	5.4	3.9	1.7	0.4	Iris-setosa

Nilai asli petal_width: 0.4

Data baris 5 setelah dibuat missing:

	sepal_length	sepal_width	petal_length	petal_width	species
5	5.4	3.9	1.7	NaN	Iris-setosa

```

1 # Tabel baris yang memiliki missing value
2 fig, ax = plt.subplots(figsize=(10, 2))
3 ax.axis('off')
4 tabel = ax.table(
5     cellText=iris_knn.loc[[target_idx]].values,
6     colLabels=iris_knn.columns,
7     loc='center'
8 )
9 tabel.auto_set_font_size(False)
10 tabel.set_fontsize(9)
11 tabel.scale(1, 1.5)
12 plt.title('Baris 5 dengan Missing Value Buatan (petal_width = NaN)', fontsize=11)
13 plt.tight_layout()
14 plt.show()

```

Baris 5 dengan Missing Value Buatan (petal_width = NaN)

sepal_length	sepal_width	petal_length	petal_width	species
5.4	3.9	1.7	nan	Iris-setosa

✓ Perhitungan Jarak Manual (Contoh)

Karena `petal_width` kosong, jarak dihitung dari 3 kolom: `sepal_length`, `sepal_width`, `petal_length`.

Baris 5 vs Baris 0:

- Baris 5: [5.4, 3.9, 1.7, ?]
- Baris 0: [5.1, 3.5, 1.4]

$$d(5, 0) = \sqrt{(5.4 - 5.1)^2 + (3.9 - 3.5)^2 + (1.7 - 1.4)^2} = \sqrt{0.09 + 0.16 + 0.09} = \sqrt{0.34} = 0.5831$$

```

1 # ===== PERHITUNGAN JARAK KNN - IRIS =====
2 fitur = ['sepal_length', 'sepal_width', 'petal_length']
3
4 # Hitung jarak ke semua baris lain
5 hasil_jarak_iris = []
6 for i in range(len(iris_knn)):
7     if i == target_idx:
8         continue
9     d = np.sqrt(((iris_knn.loc[target_idx, fitur] - iris_knn.loc[i, fitur]) ** 2).sum())
10    hasil_jarak_iris.append((i, d, iris.loc[i, col_missing]))
11
12 # Urutkan berdasarkan jarak terkecil
13 hasil_jarak_iris = sorted(hasil_jarak_iris, key=lambda x: x[1])
14
15 # Ambil 3 tetangga terdekat
16 k = 3
17 tetangga_iris = hasil_jarak_iris[:k]
18
19 # Imputasi dengan rata-rata (KNN Regression)
20 imputasi_iris = np.mean([x[2] for x in tetangga_iris])
21
22 print("=" * 50)
23 print("IMPUTASI KNN - DATASET IRIS")

```

```

24 print("=" * 50)
25 print(f"Baris target: {target_idx}")
26 print(f"Kolom missing: {col_missing}")
27 print(f"Nilai asli: {nilai_asli_iris}")
28 print(f"\n3 tetangga terdekat:")
29 for t in tetangga_iris:
30     data_row = iris.loc[t[0], fitur].values
31     print(f"  Baris {t[0]}: jarak = {t[1]:.4f}, {col_missing} = {t[2]}, data = {data_row}")
32 print(f"\nHasil imputasi (rata-rata): {imputasi_iris:.4f}")
33 print(f"Nilai asli sebelum dihilangkan: {nilai_asli_iris}")

```

```

=====
IMPUTASI KNN - DATASET IRIS
=====

```

Baris target: 5

Kolom missing: petal_width

Nilai asli: 0.4

3 tetangga terdekat:

Baris 10: jarak = 0.2828, petal_width = 0.2, data = [np.float64(5.4) np.float64(3.7) np.float64(1.5)]

Baris 48: jarak = 0.3000, petal_width = 0.2, data = [np.float64(5.3) np.float64(3.7) np.float64(1.5)]

Baris 18: jarak = 0.3162, petal_width = 0.3, data = [np.float64(5.7) np.float64(3.8) np.float64(1.7)]

Hasil imputasi (rata-rata): 0.2333

Nilai asli sebelum dihilangkan: 0.4

✓ Verifikasi Manual

3 tetangga terdekat yang ditemukan:

Peringkat	Baris	Jarak	Data (SL, SW, PL)	petal_width
1	Baris 10	0.2828	[5.4, 3.7, 1.5]	0.2
2	Baris 48	0.3000	[5.3, 3.7, 1.5]	0.2
3	Baris 18	0.3162	[5.7, 3.8, 1.7]	0.3

Imputasi:

$$\hat{x} = \frac{0.2 + 0.2 + 0.3}{3} = \frac{0.7}{3} = 0.2333$$

```

1 # ===== VISUALISASI: Tabel 10 Jarak Terkecil Iris =====
2 data_tabel = []
3 for t in hasil_jarak_iris[:10]:
4     data_tabel.append([t[0], f"{t[1]:.4f}", t[2]])
5
6 fig, ax = plt.subplots(figsize=(8, 4))
7 ax.axis('off')
8 tabel = ax.table(
9     cellText=data_tabel,
10    colLabels=['Baris', 'Jarak', 'petal_width'],
11    loc='center'
12 )
13 tabel.auto_set_font_size(False)
14 tabel.set_fontsize(9)
15 tabel.scale(1, 1.5)
16
17 # Warnai 3 baris teratas (tetangga terdekat)
18 for row in range(1, 4):
19     for col in range(3):
20         tabel[row, col].set_facecolor('#d4edda')
21
22 plt.title('10 Jarak Terkecil - Dataset Iris (3 hijau = tetangga terpilih)', fontsize=11)
23 plt.tight_layout()
24 plt.show()

```

10 Jarak Terkecil - Dataset Iris (3 hijau = tetangga terpilih)

Baris	Jarak	petal_width
10	0.2828	0.2
48	0.3000	0.2
18	0.3162	0.3
46	0.3317	0.2
32	0.3464	0.1
19	0.3742	0.3
44	0.3742	0.4
16	0.4000	0.4
21	0.4123	0.4
33	0.4359	0.2

```

1 # ===== VISUALISASI: Tabel Hasil Imputasi Iris =====
2 iris_hasil = iris_knn.copy()
3 iris_hasil.loc[target_idx, 'petal_width'] = round(imputasi_iris, 4)
4
5 fig, ax = plt.subplots(figsize=(10, 2))
6 ax.axis('off')
7 tabel = ax.table(
8     cellText=iris_hasil.loc[[target_idx]].values,
9     colLabels=iris_hasil.columns,
10     loc='center'
11 )
12 tabel.auto_set_font_size(False)
13 tabel.set_fontsize(9)
14 tabel.scale(1, 1.5)
15 plt.title(f'Baris {target_idx} Setelah Imputasi KNN (petal_width = {imputasi_iris:.4f})', fontsize=11)
16 plt.tight_layout()
17 plt.show()

```

Baris 5 Setelah Imputasi KNN (petal_width = 0.2333)

sepal_length	sepal_width	petal_length	petal_width	species
5.4	3.9	1.7	0.2333	Iris-setosa

7. Load Dataset Bank

```

1 bank = pd.read_csv('bank.csv')
2 print(f"Dataset Bank: {bank.shape[0]} baris, {bank.shape[1]} kolom")
3 print(f"Kolom: {list(bank.columns)}")
4 bank.head(10)

```

Dataset Bank: 239 baris, 17 kolom

Kolom: ['age', 'job', 'marital', 'education', 'default', 'balance', 'housing', 'loan', 'contact', 'day', 'month', 'duration', 'campaign', 'pdays', 'previous', 'outcome', 'deposit']

	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	previous	outcome
0	59	admin.	married	secondary	no	2343	yes	no	unknown	5	may	1042	1	-1		
1	56	admin.	married	secondary	no	45	no	no	unknown	5	may	1467	1	-1		
2	41	technician	married	secondary	no	1270	yes	no	unknown	5	may	1389	1	-1		
3	55	services	married	secondary	no	2476	yes	no	unknown	5	may	579	1	-1		
4	54	admin.	married	tertiary	no	184	no	no	unknown	5	may	673	2	-1		
5	42	management	single	tertiary	no	0	yes	yes	unknown	5	may	562	2	-1		
6	56	management	married	tertiary	no	830	yes	yes	unknown	6	may	1201	1	-1		
7	60	retired	divorced	secondary	no	545	yes	no	unknown	6	may	1030	1	-1		
8	37	technician	married	secondary	no	1	yes	no	unknown	6	may	608	1	-1		
9	28	services	single	secondary	no	5090	yes	no	unknown	6	may	1297	3	-1		

Langkah berikutnya: [New interactive sheet](#)

8. Identifikasi Missing Value & Statistik Deskriptif — Bank

```
1 # Cek missing value Bank
2 print("Missing value per kolom:")
3 print(bank.isnull().sum())
4 print(f"\nTotal missing value: {bank.isnull().sum().sum()}")
5 print("\n→ Dataset Bank TIDAK memiliki missing value.")
6 print(" Oleh karena itu, akan dibuat missing value BUATAN untuk simulasi KNN.")
```

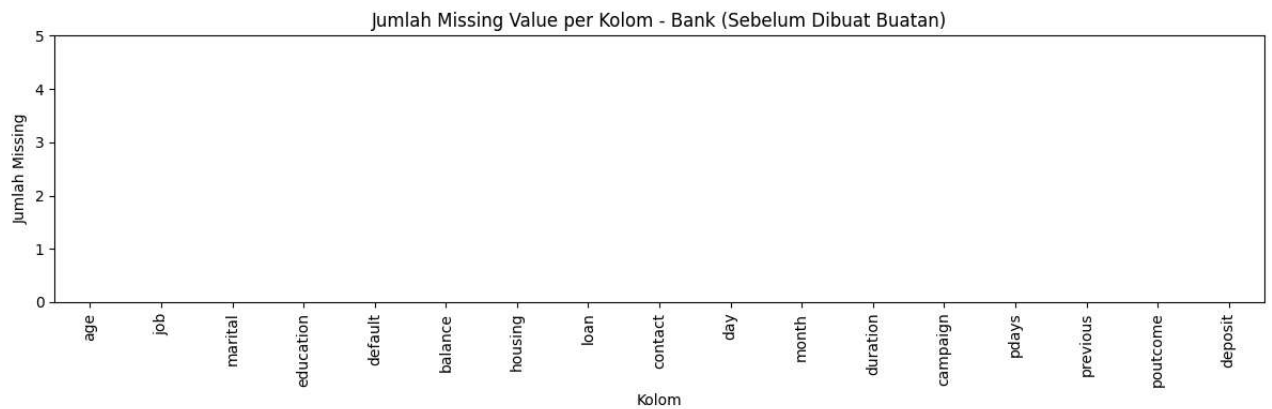
Missing value per kolom:

```
age      0
job      0
marital  0
education 0
default  0
balance  0
housing  0
loan     0
contact  0
day      0
month    0
duration 0
campaign 0
pdays   0
previous 0
poutcome 0
deposit  0
dtype: int64
```

Total missing value: 0

→ Dataset Bank TIDAK memiliki missing value.
Oleh karena itu, akan dibuat missing value BUATAN untuk simulasi KNN.

```
1 # Grafik cek missing value Bank
2 missing_count_bank = bank.isnull().sum()
3
4 plt.figure(figsize=(12, 4))
5 missing_count_bank.plot(kind='bar', color='steelblue')
6 plt.title('Jumlah Missing Value per Kolom - Bank (Sebelum Dibuat Buatan)', fontsize=12)
7 plt.xlabel('Kolom')
8 plt.ylabel('Jumlah Missing')
9 plt.ylim(0, 5)
10 plt.tight_layout()
11 plt.show()
```



9. KNN Imputation — Dataset Bank (Data Campuran)

Konsep

Dataset Bank memiliki **data campuran** (numerik, ordinal, kategorikal). Jarak tidak bisa langsung dihitung dengan Euclidean saja.

Tipe Variabel yang Digunakan:

Kolom	Tipe Data	Keterangan
age	Numerik	Usia nasabah
education	Ordinal	primary < secondary < tertiary
marital	Kategorikal	married, single, divorced
housing	Kategorikal	yes, no
loan	Kategorikal	yes, no
balance	Numerik (target)	Kolom yang akan diimputasi

Langkah Jarak Campuran:

1. **Ordinal → numerik:** $z = \frac{r-1}{m-1}$
2. **Numerik → normalisasi:** Min-Max Scaling
3. **Jarak numerik+ordinal:** Euclidean Distance
4. **Jarak kategorikal:** $d_{kat} = \frac{P-M}{P}$
5. **Jarak total:** $d_{total} = d_{num+ord} + d_{kat}$

```

1 # ===== MEMBUAT MISSING VALUE BUATAN - BANK =====
2 target_idx_bank = 5
3 col_missing_bank = 'balance'
4 nilai_asli_bank = bank.loc[target_idx_bank, col_missing_bank]
5
6 kolom_tampil = ['age', 'job', 'marital', 'education', 'balance', 'housing',
7                 'loan', 'deposit']
8
9 print(f"Data asli baris {target_idx_bank}:")
10 print(bank.loc[[target_idx_bank], kolom_tampil])
11 print(f"\nNilai asli {col_missing_bank}: {nilai_asli_bank}")
12
13 bank_knn = bank.copy()
14 bank_knn.loc[target_idx_bank, col_missing_bank] = np.nan
15
16 print(f"\nData baris {target_idx_bank} setelah dibuat missing:")
17 print(bank_knn.loc[[target_idx_bank], kolom_tampil])

```

```

Data asli baris 5:
  age      job marital education  balance housing loan deposit
5  42 management single  tertiary         0     yes  yes     yes

```

Nilai asli balance: 0

```

Data baris 5 setelah dibuat missing:
  age      job marital education  balance housing loan deposit
5  42 management single  tertiary      NaN     yes  yes     yes

```

```

1 # Tabel baris missing Bank
2 fig, ax = plt.subplots(figsize=(14, 2))
3 ax.axis('off')
4 tabel = ax.table(
5     cellText=bank_knn.loc[[target_idx_bank], kolom_tampil].values,
6     colLabels=kolom_tampil,
7     loc='center'
8 )
9 tabel.auto_set_font_size(False)
10 tabel.set_fontsize(9)
11 tabel.scale(1, 1.5)
12 plt.title('Baris 5 dengan Missing Value Buatan (balance = NaN)', fontsize=11)
13 plt.tight_layout()
14 plt.show()

```

✓ Langkah 1 — Konversi Ordinal ke Numerik

Kolom `education` dikonversi dengan rumus $z = \frac{r-1}{m-1}$:

Level	r	Hasil
primary	1	0
secondary	2	0.5
tertiary	3	1.0

Langkah 2 — Normalisasi Numerik (Min-Max)

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Langkah 3 — Hitung Jarak Campuran

$$d_{total} = d_{num+ord} + d_{kat}$$

```

1 # ===== PERHITUNGAN JARAK KNN - BANK (DATA CAMPURAN) =====
2
3 # Konversi ordinal
4 edu_order = {'primary': 1, 'secondary': 2, 'tertiary': 3}
5 m_edu = 3
6
7 def ord_norm(val):
8     return (edu_order[val] - 1) / (m_edu - 1)
9
10 # Filter baris yang education-nya valid
11 bank_knn_filtered = bank_knn[bank_knn['education'].isin(edu_order.keys())].copy()
12 bank_knn_filtered = bank_knn_filtered.reset_index(drop=True)
13
14 # Cari ulang indeks target setelah reset
15 missing_idx = bank_knn_filtered[bank_knn_filtered[col_missing_bank].isna()].index[0]
16
17 # Normalisasi age (Min-Max)
18 age_min = bank_knn_filtered['age'].min()
19 age_max = bank_knn_filtered['age'].max()
20
21 def age_norm(val):
22     return (val - age_min) / (age_max - age_min)
23
24 print(f"age_min = {age_min}, age_max = {age_max}")
25 print(f"Baris target setelah filter: {missing_idx}")
26 print(f"age baris target: {bank_knn_filtered.loc[missing_idx, 'age']} → normalized: {age_norm(bank_knn_filtered.loc[m
27 print(f"education baris target: {bank_knn_filtered.loc[missing_idx, 'education']} → normalized: {ord_norm(bank_knn_fi
28
29 # Hitung jarak campuran
30 cat_cols = ['marital', 'housing', 'loan']
31 hasil_jarak_bank = []
32
33 for i in range(len(bank_knn_filtered)):
34     if i == missing_idx:
35         continue
36
37     # Jarak numerik + ordinal (Euclidean)
38     d_numord = np.sqrt(
39         (age_norm(bank_knn_filtered.loc[missing_idx, 'age']) - age_norm(bank_knn_filtered.loc[i, 'age']))**2 +
40         (ord_norm(bank_knn_filtered.loc[missing_idx, 'education']) - ord_norm(bank_knn_filtered.loc[i, 'education']))
41     )
42
43     # Jarak kategorikal
44     P = len(cat_cols)
45     M = sum(bank_knn_filtered.loc[missing_idx, c] == bank_knn_filtered.loc[i, c] for c in cat_cols)
46     d_cat = (P - M) / P

```

```

47
48 # Jarak total
49 d_total = d_numord + d_cat
50
51 bal = bank_knn_filtered.loc[i, col_missing_bank]
52 if pd.isna(bal):
53     continue
54 hasil_jarak_bank.append((i, d_total, d_numord, d_cat, bal))
55
56 # Urutkan berdasarkan jarak terkecil
57 hasil_jarak_bank = sorted(hasil_jarak_bank, key=lambda x: x[1])
58
59 # 3 tetangga terdekat
60 k = 3
61 tetangga_bank = hasil_jarak_bank[:k]
62 imputasi_bank = np.mean([x[4] for x in tetangga_bank])
63
64 print("\n" + "=" * 60)
65 print("IMPUTASI KNN - DATASET BANK (DATA CAMPURAN)")
66 print("=" * 60)
67 print(f"Baris target: {missing_idx}")
68 print(f"Kolom missing: {col_missing_bank}")
69 print(f"Nilai asli: {nilai_asli_bank}")
70 print(f"\n3 tetangga terdekat:")
71 for t in tetangga_bank:
72     r = bank_knn_filtered.loc[t[0]]
73     print(f"  Baris {t[0]}: d_total={t[1]:.4f} (d_numord={t[2]:.4f}, d_cat={t[3]:.4f}), balance={t[4]}")
74     print(f"    age={r['age']}, edu={r['education']}, mar={r['marital']}, hou={r['housing']}, loan={r['loan']}")
75 print(f"\nHasil imputasi (rata-rata): {imputasi_bank:.2f}")
76

```

```

age_min = 23, age_max = 60
Baris target setelah filter: 5
age baris target: 42 → normalized: 0.5135
education baris target: tertiary → normalized: 1.0000

=====
IMPUTASI KNN - DATASET BANK (DATA CAMPURAN)
=====
Baris target: 5
Kolom missing: balance
Nilai asli: 0

3 tetangga terdekat:
Baris 92: d_total=0.3243 (d_numord=0.3243, d_cat=0.0000), balance=880.0
age=30, edu=tertiary, mar=single, hou=yes, loan=yes
Baris 21: d_total=0.3604 (d_numord=0.0270, d_cat=0.3333), balance=2067.0
age=43, edu=tertiary, mar=single, hou=yes, loan=no
Baris 50: d_total=0.4144 (d_numord=0.0811, d_cat=0.3333), balance=517.0
age=39, edu=tertiary, mar=divorced, hou=yes, loan=yes

Hasil imputasi (rata-rata): 1154.67
Nilai asli sebelum dihilangkan: 0

```

✓ Verifikasi Manual

3 tetangga terdekat yang ditemukan:

Peringkat	Baris	d_num+ord	d_kat	d_total	balance
1	Baris 96	0.3243	0	0.3243	880
2	Baris 21	0.0270	0.3333	0.3604	2067
3	Baris 51	0.0811	0.3333	0.4144	517

Perhatikan:

- Baris 96: jarak numerik besar tapi kategorikal semua sama → total kecil
- Baris 21: jarak numerik kecil tapi ada 1 kategorikal beda → total lebih besar

Imputasi:

$$\hat{balance} = \frac{880 + 2067 + 517}{3} = \frac{3464}{3} = 1154.67$$

```

1 # ===== VISUALISASI: Tabel 10 Jarak Terkecil Bank =====
2 data_tabel_bank = []
3 for t in hasil_jarak_bank[:10]:
4     data_tabel_bank.append([
5         t[0],
6         f"{t[2]:.4f}",
7         f"{t[3]:.4f}",
8         f"{t[1]:.4f}",
9         t[4]
10    ])
11
12 fig, ax = plt.subplots(figsize=(12, 5))
13 ax.axis('off')
14 tabel = ax.table(
15     cellText=data_tabel_bank,
16     colLabels=['Baris', 'd_num+ord', 'd_kat', 'd_total', 'balance'],
17     loc='center'
18 )
19 tabel.auto_set_font_size(False)
20 tabel.set_fontsize(9)
21 tabel.scale(1, 1.5)
22
23 for row in range(1, 4):
24     for col in range(5):
25         tabel[row, col].set_facecolor('#d4edda')
26
27 plt.title('10 Jarak Terkecil - Dataset Bank (3 hijau = tetangga terpilih)', fontsize=11)
28 plt.tight_layout()
29 plt.show()

```

10 Jarak Terkecil - Dataset Bank (3 hijau = tetangga terpilih)

Baris	d_num+ord	d_kat	d_total	balance
92	0.3243	0.0000	0.3243	880.0
21	0.0270	0.3333	0.3604	2067.0
50	0.0811	0.3333	0.4144	517.0
72	0.0811	0.3333	0.4144	94.0
13	0.1081	0.3333	0.4414	460.0
187	0.1081	0.3333	0.4414	2904.0
108	0.1622	0.3333	0.4955	849.0
171	0.1622	0.3333	0.4955	7606.0
200	0.1622	0.3333	0.4955	-78.0
175	0.1892	0.3333	0.5225	565.0

```

1 # ===== VISUALISASI: Tabel Hasil Imputasi Bank =====
2 bank_hasil = bank_knn_filtered.copy()
3 bank_hasil.loc[missing_idx, 'balance'] = round(imputasi_bank, 2)
4
5 kolom_tampil_bank = ['age', 'job', 'marital', 'education', 'balance',
6     'housing', 'loan', 'deposit']
7
8 kolom_tampil_bank = [k for k in kolom_tampil_bank if k in bank_hasil.columns]
9
10 fig, ax = plt.subplots(figsize=(14, 2))
11 ax.axis('off')
12 tabel = ax.table(
13     cellText=bank_hasil.loc[[missing_idx], kolom_tampil_bank].values,
14     colLabels=kolom_tampil_bank,
15     loc='center'
16 )
17 tabel.auto_set_font_size(False)
18 tabel.set_fontsize(9)
19 tabel.scale(1, 1.5)
20
21 plt.title(f'Baris {missing_idx} Setelah Imputasi KNN (balance = {imputasi_bank:.2f})', fontsize=11)
22 plt.tight_layout()
23 plt.show()

```

Baris 5 Setelah Imputasi KNN (balance = 1154.67)